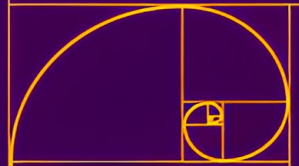
HUNTER
The City University of New York

# Algorithm Analysis

## CSCI 33500: Software Design and Analysis III

**Sadab Hafiz**
sh3646@hunter.cuny.edu



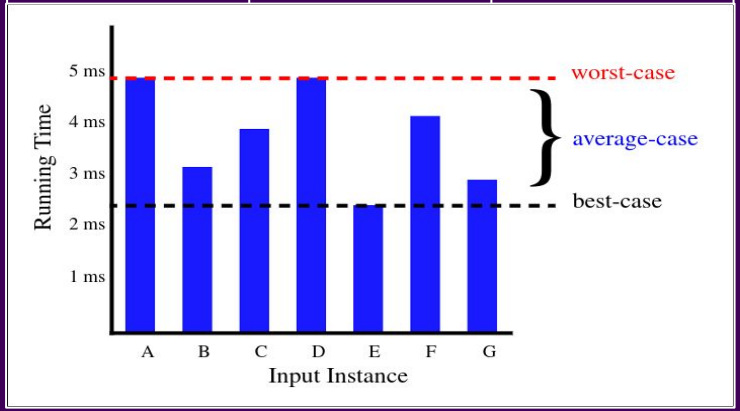
02

# Recap: Big-O Notation

- German mathematician Paul Bachmann first introduced the O-notation in 1894
- Edmund Laudau popularized the notation in the early 1900s
- For this reason, the family of asymptotic symbols (O, o, Ω, ω, Θ) is often called Landau notation
- The “O” stands for the German word “Ordnung”, which can be translated as “order” (in the sense of growth order).

Source: https://en.wikipedia.org/wiki/Big_O_notation

| Notation | Complexity | Bound |
|---|---|---|
| **Big-O** | Worst Case | Upper |
| **Big-Ω** | Best Case | Lower |
| **Big-Θ** | Average Case | Tight |



# What is little-ω?

- Little-ω (omega) means f grows strictly faster than g. There is no constant multiple of g that eventually bounds f from above:

$$f(N) = \omega(g(N)) \iff \lim_{N \to \infty} \frac{f(N)}{g(N)} = \infty$$

- For **every** constant $c > 0$, there exists a constant $N_0 > 0$ such that, for all $N \geq N_0$: $f(N) > c \cdot g(N)$
- Little-ω implies big-Ω because little-ω is a stronger lower-bound
- Little-ω is the strict opposite of little-o
- Example: $N^2 = \omega(N)$

Can you prove the example?

# All Five Symbols

## Landau Symbols Summary

Source: Algorithms and Data Structures | University of Waterloo

$$f(n) = o(g(n)) \qquad \lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$$

$$f(n) = O(g(n)) \qquad \lim_{n\to\infty} \frac{f(n)}{g(n)} < \infty$$

$$f(n) = \Theta(g(n)) \qquad 0 < \lim_{n\to\infty} \frac{f(n)}{g(n)} < \infty$$

$$f(n) = \Omega(g(n)) \qquad \lim_{n\to\infty} \frac{f(n)}{g(n)} > 0$$

$$f(n) = \omega(g(n)) \qquad \lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty$$

We say $f(n) =$ $O(g(n))$ $\Omega(g(n))$ $o(g(n))$ $\Theta(g(n))$ $\omega(g(n))$

if $\lim_{n\to\infty} \frac{f(n)}{g(n)} =$ $0$ $0 < c < \infty$ $\infty$

# Additional Notes on Big-O Notation

- Algorithm speed vs growth rate of function:
  - faster asymptotic growth rate = higher runtime = slower algorithm
  - slower asymptotic growth rate = lower runtime = faster algorithm
  - O(n) can be described as “faster than $O(n^2)$” but also “slower growing than $O(n^2)$”
- Logarithmic, polynomial, and exponential functions
  - form a strict hierarchy (Logarithmic < Linear < Quadratic < ··· < Polynomial < Exponential)
  - every asymptotic complexity class is either o, ω, or Θ of each other
  - Big-O with a tight-bound is the same as Big-Θ
  - the above points are not always true for other type of functions (periodic, piecewise, etc.)
- We typically use Big-O with a tight bound (which is usually implied)
  - While it would be technically correct to say that merge sort is $O(n^2)$, it can be misleading
  - You would only say that merge sort is O(n log n)
    - unless you're in the middle of a proof establishing a bound

# Operators

Θ(1)

Source: Algorithms and Data Structures | University of Waterloo

- Memory Operations
  - Retrieving/Storing
  - Memory allocation: `new`
  - Memory deallocation: `delete`
- Variable Assignment (=)
- Integer Operations
  - `+ - * / % ++ −`
- Logical Operations
  - and(&&)
  - or(||)
  - not(!)
- Bitwise operations
  - Bitwise AND (&)
  - Bitwise OR (|)
  - Bitwise XOR (^)
  - Bitwise NOT (~)
- Relational operations
  - `== != < <= => >`

# Operators

Memory Considerations

Source: Algorithms and Data Structures | University of Waterloo

- ➔ Memory allocation and deallocation are the slowest by a significant factor compared to other operators
- ➔ Can be slow by a factor of over 100 compared to other operators
- ➔ They require communication with the operating system
- ➔ This does not account for the time required to call the constructor and the destructor.
- ➔ After memory is allocated, the constructor is run. The constructor itself may not run in $\Theta(1)$ time.

# Control Statements

Statements That Potentially Alter Execution of Instructions

Source: Algorithms and Data Structures | University of Waterloo

- Conditional Statements
  - `if`, `switch`
- Condition-controlled Loops
  - `for`, `while`, `do-while`
- Count-controlled loops
  - `for i in range(5,10) # python`
- Collection-controlled loops
  - `for (int num : numbers)`
- Given any collection of nested control statements, it is always necessary to work inside out
  - Determine runtimes of the innermost statements and work your way out

# Conditionals

Control Statements

Source: Algorithms and Data Structures | University of Waterloo

- Given:

```
if ( condition ) {
    // true body
} else {
    // false body
}
```

- The runtime of a conditional statement is the sum of:
  - The runtime of the condition (the test)
  - The runtime of the body which is run
- In most cases, the runtime of the condition inside `if()` is Θ(1)

# Conditionals

Examples

Source: Algorithms and Data Structures | University of Waterloo

It is easy to determine in some cases:

```
int factorial ( int n ) {
        if ( n == 0 ) {
                return 1;
        } else {
                return n * factorial ( n - 1 );
        }
}
```

Less obvious in other cases:

```
int find_max( int *array, int n ) {
    max = array[0];

    for ( int i = 1; i < n; ++i ) {
        if ( array[i] > max ) {
            max = array[i];
        }
    }

    return max;
}
```

```
int find_max( int *array, int n ) {
    max = array[0];

    for ( int i = 1; i < n; ++i ) {
        if ( array[i] > max ) {
            max = array[i];
        }
    }

    return max;
}
```

**What is the Big-O time complexity?**

- ➔ In this case, we don’t know
- ➔ If we had information about the distribution of the entries, we may be able to determine it:
  - What if the array is sorted (ascending)?
  - What if the array is sorted (descending)?
  - uniform random distribution???

# Loops

Condition Controlled

**Why is the for-loop “condition controlled”?**

➔ C++ `for`-loop:

```
for ( int i = 0; i < N; ++i ) {
        // ...
}
```

➔ C++ `while`-loop:

```
int i = 0;                  // initialization
while ( i < N ) {           // condition
    // ...
    ++i;                    // increment
}
```

➔ The above loops are identical!

# For-Loops

Time Complexity

Source: Algorithms and Data Structures | University of Waterloo

- The initialization, condition, and increment are usually single statements running in $\Theta(1)$
- Assuming there are no `break` or `return` statements in the loop, the runtime is $\Omega(n)$

```
for ( int i = 0; i < n; ++i ) {
    // code which is Theta(f(n))
}
```

- If the body does not depend on the variable (above example): $\Theta(n \cdot f(n))$
- If the body is $O(f(n))$, then the runtime of the loop is $O(n \cdot f(n))$

# Loops

Examples

Source: Algorithms and Data Structures | University of Waterloo

```
int sum = 0;
for ( int i = 0; i < n; ++i ) {
    sum += 1;      Theta(1)
}
```

➔ The runtime for this is $\Theta(n \cdot 1) = \Theta(n)$

```
int sum = 0;
for ( int i = 0; i < n; ++i ) {
    for ( int j = 0; j < n; ++j ) {
        sum += 1;      Theta(1)
    }
}
```

➔ We know the inner-loop is $\Theta(n)$.
Thus, the outer-loop is $\Theta(n \cdot n) = \Theta(n^2)$

# Loops

More Examples

Source: Algorithms and Data Structures | University of Waterloo

```
for ( int i = 0; i < n; ++i ) {
        // search through an array of size m
        // O( m );
}
```

➔ The inner-loop is O(m) and thus the outer-loop is O(n·m)

```
int sum = 0;
for ( int i = 0; i < n; ++i ) {
    for ( int j = 0; j < i; ++j ) {
        sum += i + j;
    }
}
```

What about this one?

# Body Depends on Loop Variable

```
int sum = 0;
for ( int i = 0; i < n; ++i ) {
    for ( int j = 0; j < i; ++j ) {
        sum += i + j;
    }
}
```

Source: Algorithms and Data Structures | University of Waterloo

➔ If the **body depends on the loop variable** (in below example, `i`), then the runtime of

```
for ( int i = 0; i < n; ++i ) {
    // code which is Theta(f(i,n))
}
```

is $\Theta\left(1+\sum_{i=0}^{n-1}(1+f(i,n))\right)$ and if the body is $\mathbf{O}(f(i,n))$, the result is

$$O\left(1+\sum_{i=0}^{n-1}(1+f(i,n))\right)$$

# Runtime of the Inner Loop

```
int sum = 0;
for ( int i = 0; i < n; ++i ) {
    for ( int j = 0; j < i; ++j ) {
        sum += i + j;
    }
}
```

- ➔ The loop initialization is 1 operation
- ➔ Loop repeats `i` number of times
- ➔ `i + j` is 1 operation inside loop
- ➔ `sum +=` is 1 operation inside loop
- ➔ Thus, inner-loop: Θ(1 + `i`(1 + 1))
- ➔ Dropping the constants, Θ(`i`)

# Runtime of the Outer Loop

```
int sum = 0;
for ( int i = 0; i < n; ++i ) {
    for ( int j = 0; j < i; ++j ) {
        sum += i + j;
    }
}
```

$$O\left(1+\sum_{i=0}^{n-1}(1+f(i,n))\right)$$

- The variable `i` goes from 0 to n
- Recall, $T_1(N) + T_2(N) = O(f(N) + g(N))$
  - Applies to all Big-O notation
- Thus, 0 + 1 + 2 + 3 + . . . + n
- In each iteration, we have Θ(`i`) loop
- Using the above, we can say:

$$f(i,n)=\sum_{i=0}^{n-1} i$$

Did we see this before?

# Arithmetic Series Proof

$$0+1+2+3+\cdots+n=\sum_{k=0}^{n}k=\frac{n(n+1)}{2}$$

➔ Write out the series twice and add each column together

$$\begin{array}{ccccccccccccc} & 1 & + & 2 & + & 3 & + \cdots + & n-2 & + & n-1 & + & n \\ + & n & + & n-1 & + & n-2 & + \cdots + & 3 & + & 2 & + & 1 \\ \hline & (n+1) & + & (n+1) & + & (n+1) & + \cdots + & (n+1) & + & (n+1) & + & (n+1) \end{array}$$

➔ Result: $n\,(n+1)$

➔ Since we added the series to itself, we have to divide the result with 2

Source: Algorithms and Data Structures | University of Waterloo

# Proof by Induction

$$0+1+2+3+\cdots+n=\sum_{k=0}^{n}k=\frac{n(n+1)}{2}$$

- ➔ Base Case:
  - The statement is true for n = 0:

$$\sum_{i=0}^{0}k=0=\frac{0\cdot 1}{2}=\frac{0(0+1)}{2}$$

- ➔ Inductive Hypothesis:
  - Assume that the statement is true for an arbitrary *m* (where *m* ≥ 0)

$$\sum_{k=0}^{m}k=\frac{m(m+1)}{2}$$

- ➔ Inductive Step:
  - for *n* = *m* + 1, we must show that:

$$\sum_{k=0}^{m+1}k=\frac{(m+1)(m+2)}{2}$$

# Inductive Step

$$0+1+2+3+\cdots+n=\sum_{k=0}^{n}k=\frac{n(n+1)}{2}$$

➔ Inductive Step:

$$\sum_{k=0}^{m+1}k=\frac{(m+1)(m+2)}{2}$$

$$=(m+1)+\frac{m(m+1)}{2}$$

$$=\frac{(m+1)2+(m+1)m}{2}$$

$$=\frac{(m+1)(m+2)}{2}$$

# Runtime of the Outer Loop

```
int sum = 0;
for ( int i = 0; i < n; ++i ) {
    for ( int j = 0; j < i; ++j ) {
        sum += i + j;
    }
}
```

$$O\left(1+\sum_{i=0}^{n-1}(1+f(i,n))\right)$$

$$f(i,n)=\sum_{i=0}^{n-1} i$$

➔ Thus, using the Arithmetic Series:

$$\Theta\left(1+\sum_{i=0}^{n-1}(i+1)\right)$$

$$=\Theta\left(1+\sum_{i=0}^{n-1} i+\sum_{i=0}^{n-1} 1\right)$$

$$=\Theta\left(1+\frac{n(n-1)}{2}+n\right)$$

$$=\Theta(n^2)$$

# Nested Loop

Example

Source: Algorithms and Data Structures | University of Waterloo

```
int sum = 0;
for ( int i = 0; i < n; ++i ) {
    for ( int j = 0; j < i; ++j ) {
        for ( int k = 0; k < j; ++k ) {
            sum += i + j + k;
        }
    }
}
```

- From inside to out:
  - Inner most `sum +=` : $\Theta(1)$
  - Inner most loop (`int k`): $\Theta(j)$
  - 2nd nested loop (`int j`: $\Theta(i^2)$
  - Outer loop (`int i`): $\Theta(n^3)$
- Thus, $\Theta(n^3)$

# Conditional Statements Example

```
void Disjoint_sets::clear() {
    if ( sets == n ) {
        return;                          Θ(1)
    }

    max_height = 0;
    num_disjoint_sets = n;               Θ(1)

    for ( int i = 0; i < n; ++i ) {
        parent[i] = i;
        tree_height[i] = 0;              Θ(n)
    }
}     Θ(1)
```

$$T_{clear}(n) = \begin{cases} \Theta(1) & sets = n \\ \Theta(n) & \text{otherwise} \end{cases}$$

Source: Algorithms and Data Structures | University of Waterloo

# Conditionals: Switch Statements and If Statements

➔ Switch statements appear to be nested if-statements:

```
switch( i ) {
       case 1:   /* do stuff */ break;
       case 2:   /* do other stuff */ break;
       case 3:   /* do even more stuff */ break;
       case 4:   /* well, do stuff */ break;
       case 5:   /* tired yet? */ break;
       default:  /* do default stuff */
}
```

```
if ( i == 1 ) { /* do stuff */ }
else if ( i == 2 ) { /* do other stuff */ }
else if ( i == 3 ) { /* do even more stuff */ }
else if ( i == 4 ) { /* well, do stuff */ }
else if ( i == 5 ) { /* tired yet? */ }
else { /* do default stuff */ }
```

➔ A switch statement would appear to run in O(n) time, where *n* is the number of cases, the same as nested if statements

➔ Why do switch statements exist then?

Source: Algorithms and Data Structures | University of Waterloo

# Why switch?

The switch statement

Source: Algorithms and Data Structures | University of Waterloo

- ➔ Switch statements were included in the original C language, but why?
- ➔ First, recall that unlike if statements, the cases **must be actual values**, integral and enumeration types:
  - integers:
    - `int`, `short`, `long`, etc.
  - Characters
    - `char`, `wchar_t`, `char16_t`, etc.
  - Booleans (rarely done)
  - Enumerations: Standard `enum` and scoped enum class types. This is one of the most common and powerful uses.
- ➔ Cannot have a case with a variable!

# Why switch?

Compiler Jump

- The compiler looks at the different cases and calculates an appropriate jump using a jump table (array of memory addresses)
- For example, assume:
  - The cases are 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
  - The compiler stores the memory address of each
- Instead of checking each condition one by one, the compiler simply multiplies the variable by the size of a memory pointer to jump directly to the correct address
- Using  a single mathematical calculation to find the destination instantly, an optimized switch statement runs in O(1) time, regardless of the number of statements

# Summary

Rules from Textbook (2.4.2)

- Rule 1 - FOR loops
  - The running time of a for loop is at most the running time of the statements inside the for loop (including tests) times the number of iterations.
- Rule 2 - Nested loops
  - Analyze these inside out. The total running time of a statement inside a group of nested loops is the running time of the statement multiplied by the product of the sizes of all the loops.
- Rule 3 - Consecutive Statements
  - These just add (which means that the maximum is the one that counts)
- Rule 4 - If/Else
  - The running time of an if/else statement is never more than the running time of the test plus the larger of the running times of S1 and S2.